Personalized AI Chatbot System Supports Self-Reflection

I. Abstract

In the digital age, the need for **self-reflection and self-development** is increasing, but existing tools have not met the personalization **according** to each user's style. This project proposes a **personalized AI chatbot system that** supports users in self-reflection, by realistically simulating their own thinking and expression style. The chatbot is built on a large language model platform (GPT-4) combined with the user's **persona profile and a long-term conversational memory** mechanism (via a vector database) to maintain the context throughout. The system follows a microservices architecture, including a mobile application with a chat interface, a backend that integrates the GPT-4 API, a SQL/NoSQL hybrid database for user and conversation data, and a **CBT (Cognitive Behavioral Therapy) feedback analysis module** to provide positive psychological suggestions when needed. The implementation results show that the chatbot is capable of responding in a similar style to the user, remembering information over multiple chat sessions, and supporting multiple platforms (Android, iOS). The system has met the set criteria (style similarity ≥ 85%, almost instantaneous response), opening up a new approach in combining AI and psychology to create a digital **"companion"** to help users reflect and improve themselves.

II. Introduction

2.1 Motivation

In **the digital age , the need for self-development and mental health care has become especially important. Users want effective self-reflection** tools that help them review their thoughts and feelings to improve their thinking. However, there is currently no AI chatbot that can accurately reflect each individual 's **unique thinking style, values, and emotions** . Most existing virtual assistants use a **fixed personality** and a templated response script for every user, lacking the ability to customize. This leads to superficial interactions that fail to create deep connections and trust with users.

The current landscape shows the intersection of AI technology and psychology. Many **mental health chatbots** have emerged as a new approach, combining **artificial intelligence** with psychological therapies. For example, *Woebot* is a chatbot that uses cognitive behavioral therapy (CBT) to help young users reduce depression and anxiety. Recent surveys show that there are more than **50 mental health support chatbots** with various goals (general psychological support, reducing depression, anxiety, stress, etc.). These systems open up a convenient approach, allowing psychological support anytime, anywhere, reducing the workload for professionals and removing the cost and time barriers compared to traditional therapy. **However** , most current chatbots still have a very limited level of personalization – they often do not maintain long-term conversational context and do not adapt flexibly to each user's **preferences, habits or values .** For example, large general language models like ChatGPT or Bard, while powerful, are “one size fits all”: they rely on user-provided prompts to play the desired role, which is both inconvenient and unnatural for the average user. Furthermore, these models are limited by context length, making them prone to forgetting details from previous conversations. Users may encounter situations where the chatbot **“forgets”** information they shared in a previous session, making the conversation lose its necessary continuity and personalization. These limitations make it difficult for chatbots to be a trusted companion in each individual’s journey of self-reflection and personal growth.

Based on that reality, this project was proposed to answer the question: **how to design an AI chatbot that can “play the role” of the user, realistically simulating the way they think and express themselves, thereby supporting them to self-reflect and develop positive thinking? This is the core motivation – creating a highly personalized** chatbot system to help each user feel like they are talking to themselves, thereby recognizing problems and improving themselves.

2.2 Objectives

The project's goal is to build a **personalized AI chatbot** for self-reflection, with specific SMART criteria as follows:

- **Personal Style Simulation:** The chatbot is able to mimic the user's writing style and emotions with a **semantic similarity of ≥ 85%** compared to their expressions (measured by cosine similarity on the sentence representation

vector). This ensures that the chatbot's responses have the personal "color" of each user.

- **Long-term contextual memory:** The system maintains conversational context **across multiple** chat sessions, helping the chatbot **remember** important information that the user has previously shared and respond in a coherent and consistent manner.
- **Multi-platform support:** Chatbot applications can be deployed on **multiple platforms** (Android, iOS mobile phones, and can be extended to the web) for users to easily access anytime, anywhere.
- **Personalize via persona profiles:** Build **a persona profile** for each user (including information about values, interests, language habits, etc.) and use this profile to customize responses for them. Integrate a **basic CBT counseling module** so that the chatbot can provide positive psychological suggestions or therapeutic questions when detecting negative emotions in the user.
- **Time & Resources:** Complete development of a fully functional prototype within **16 weeks** with a team of 2–3 members. The project focuses on integrating and using available language models via API instead of training new models from scratch, to optimize time and resources. Complex features that are out of scope (e.g., intensive therapy or specialized knowledge such as medical, legal, etc.) will not be implemented in this version.

2.3 Research Questions

Based on the objectives and problems, the project aims to answer the following main research questions:

1. **How can chatbots emulate a user's style and persona?** What techniques are needed to effectively collect and apply persona profiles to language model responses, to make the chatbot feel like it has **a "personality"** similar to the user?
2. **How to maintain long-term conversational memory for chatbots?** What methods allow the system to effectively **remember and retrieve information from previous conversation sessions, overcoming the short-term context limitations of AI models?**
3. **Is integrating psychological cueing (CBT) into personalized chatbots effective?** How are sentiment analysis and CBT feedback techniques integrated into chatbot conversations, and how do they impact the **user experience** (e.g., helping users think more positively, feel more empathetic).

The above questions guide the system design and implementation, and serve as a basis for evaluating the effectiveness of the proposed solution compared to a conventional chatbot.

2.4 Scope

The project focuses on developing a **complete prototype** of the proposed chatbot system, with all the core functionalities outlined. *The scope of* work includes

architectural design, frontend development (mobile chat application), backend microservices development, integration of the GPT-4 language model via API, database development (SQL, NoSQL, vector) for conversational memory, and integration of a basic CBT sentiment analysis module. Training **a new AI model** is out of scope – the system will use existing language models and libraries (OpenAI GPT-4, SBERT, etc.). In addition, advanced functionalities that require deep expertise (such as professional psychotherapy, medical diagnosis, specific legal) are **not** within the scope of this release. The final product will be a cross-platform chatbot application running as a pilot, to demonstrate the feasibility of the idea of personalizing a chatbot to support self-reflection. Evaluation will be based on established criteria (user style similarity, memorability, response speed, etc.), through technical testing and initial feedback from test users.

III. Methodology

3.1 Theoretical Basis

The project combines advances in **Natural Language Processing (NLP)** , **machine learning** , and **psychology** to create a personalized chatbot system. In terms of NLP, the development of **Large Language Models (LLMs)** such as GPT-3.5, GPT-4 has enabled chatbots to generate natural, coherent responses in a variety of contexts. These models can understand language at a complex level and generate text that is close to human-like. However, to effectively apply LLMs to each individual, a mechanism **for adjusting the input is needed** . The project uses **Prompt Engineering** and **Few-shot Learning techniques** – which provide the model with some instructions and examples of the user's writing style right in the prompt – to guide the LLM to respond correctly to the target **persona . In addition, the " Custom ChatGPT " (OpenAI Custom Instructions)** feature can be leveraged to pre-define virtual assistants with some unique personality traits for each user.

Another important pillar is the concept of **chatbot personalization** . Recent studies have shown that when users interact with chatbots that look like themselves, they tend to be more emotionally attached and trust the system than with regular chatbots. This requires the AI model to **understand** the user's profile – including personality, interests, and speaking style – and **reflect** that in the response. The project builds **a persona profile** for each user based on the data they provide (registration questions, personal posts, chat history, etc.). This profile acts as a unique **knowledge base** for the chatbot when interacting with that user.

To make the chatbot effective in supporting self-reflection, the project integrates elements from **cognitive behavioral therapy (CBT)** – a psychotherapy method that helps correct negative and distorted thoughts. Technically, the system installs a simple **sentiment analysis module** to monitor the user's psychological state. When the chatbot detects that the user is expressing negative emotions (e.g. sadness, disappointment, negative thoughts), it automatically inserts **CBT-style suggestions into the conversation** . For example, the chatbot can ask Socratic questioning to help the user see the problem from a different perspective, or encourage them to think more positively. Studies on the use of LLM in therapy support show that the model can follow

the scenario of asking open questions and giving empathetic responses, acting as a basic “CBT assistant”. However, this integration is carefully designed to not lose the chatbot's **personal “voice”** – meaning that therapy advice or questions must be delivered in the user's own voice.

Finally, the problem of **long-term conversational memory** is solved by applying the **RAG (Retrieval-Augmented Generation) method** . Instead of relying entirely on the short-term memory of the LLM model, the system uses a vector database to store **old conversations** as embedded vectors. Every time a user sends a message, the system searches the vector space to **retrieve the most relevant conversations** from the past and provides them to the LLM as part of the input context. This technique allows the chatbot to “remember” important information from thousands of previous interactions without exceeding the model’s context limit. Algorithmically, the solution utilizes **Sentence-BERT (SBERT)** to convert sentences into semantic vectors – sentences with similar content will have vectors close to each other in space. Thanks to that, with just a **cosine similarity measure** , the system can find old sentences/conversations related to the current topic to supplement the context. This is the platform that helps chatbots maintain **continuity and consistency** in long-term dialogue with users.

3.2 Architecture Design

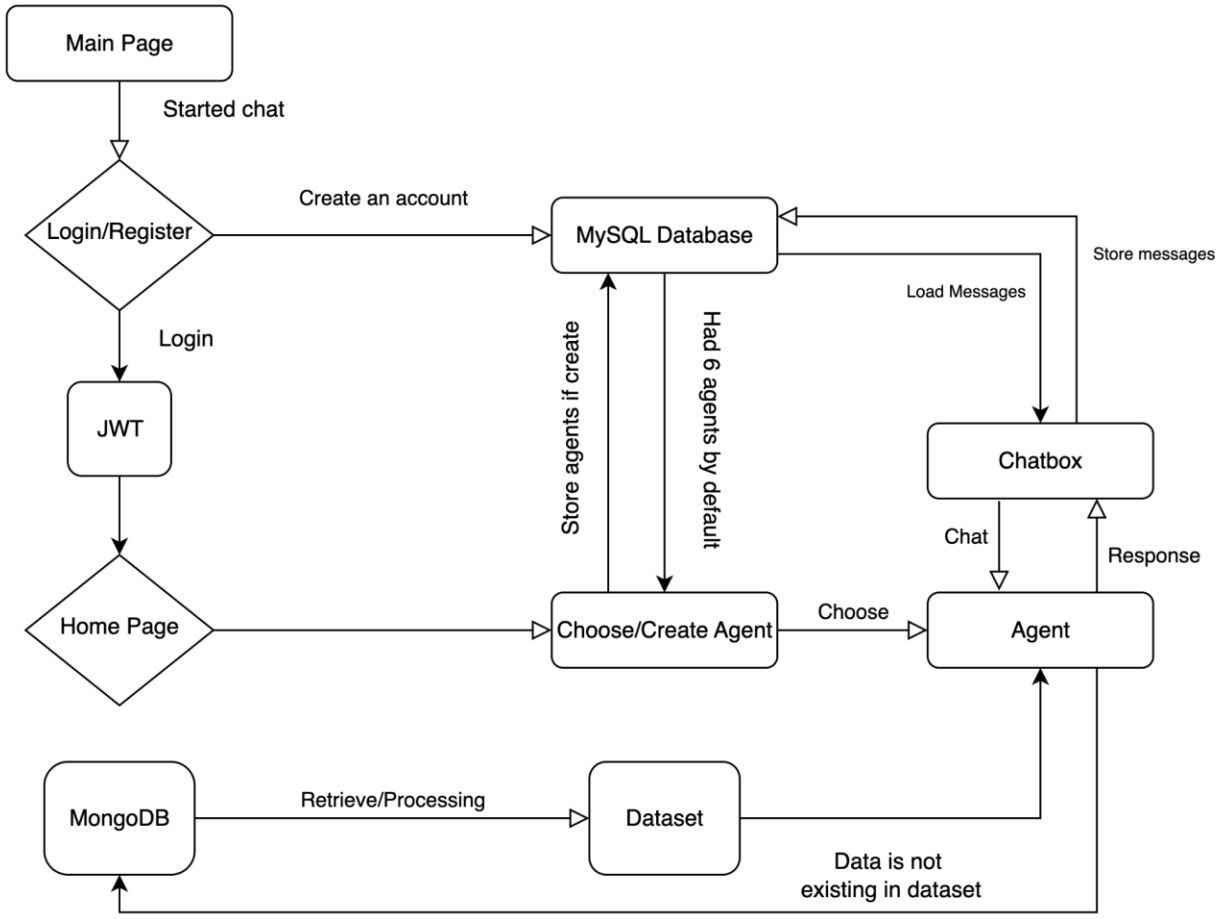
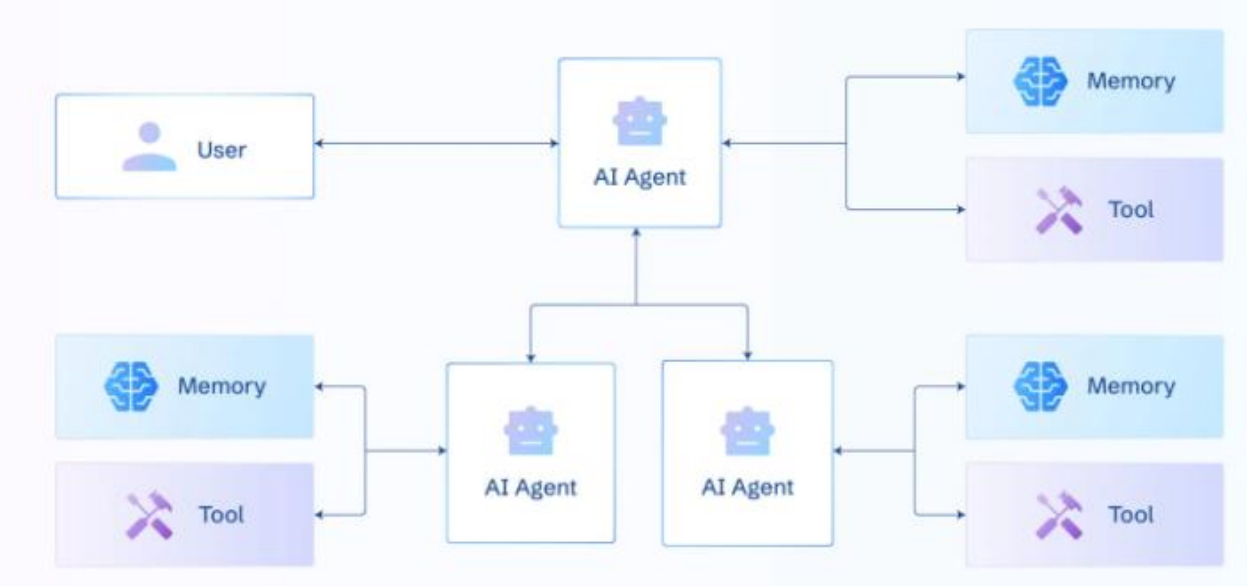


Diagram Workflow Overall



AI Agent Diagram

microservices architecture to ensure flexibility and future scalability. The overall architecture includes the following main components:

- **Frontend (Client Application):** The user interface is developed as a cross-platform mobile application (Android, iOS) using Flutter. This application provides a friendly chat interface for users to interact with the chatbot. It is responsible for displaying the conversation, sending user messages to the server, and receiving and displaying responses from the chatbot in real time. (In the future, the interface can be extended to the web platform if necessary.)
- **Backend (Service API):** The backend server is built with Node.js and the Express.js framework, providing RESTful APIs as well as WebSocket endpoints for real-time chat functionality. The backend is divided into many **small services** (microservices), each of which undertakes a specific function and can be deployed independently. The main services include: user management service (registration, login, persona profile management), conversation service (receiving messages, managing chat sessions), LLM integration service (sending requests to GPT-4 API and receiving responses), CBT feedback processing & consulting service (analyzing message sentiments and adding suggestions if needed), similarity measurement service (calculating the similarity between AI responses and user writing style), etc. The services communicate with each other via internal HTTP or via message queue to ensure scalability and **maintainability** . The microservices architecture allows upgrading or replacing individual components without affecting the entire system.
- **Database:** The system uses a combination of both relational (SQL) and **NoSQL databases** to take advantage of the advantages of each type. Specifically, **PostgreSQL** is used to store important data with clear structures such as user

account information, application configuration, login history, user reviews, etc. Using PostgreSQL ensures integrity (ACID) and is convenient when complex queries are needed using SQL. In parallel, **MongoDB** - a document-based NoSQL database - is used to store conversation content and persona profiles as flexible JSON documents. Each user can have a different amount of profile information and conversation logs, so MongoDB is very suitable thanks to its flexible, scalable schema. The combination of SQL + NoSQL helps the system to be both **tight** in core data and **flexible** in storing rich chat content and user characteristics. In addition, the system also integrates **Redis** as an in-memory data store for caching: Redis temporarily stores recent conversation sessions as a **short-term memory** , helping to speed up retrieval of the latest context and reduce the load on the main database. Finally, a **Vector Database** (e.g. Pinecone service, Milvus, or self-implemented FAISS library) is used to store embedded vectors for long-term conversation memory. Each time a user sends a message, the backend will create a vector representing that sentence (using the SBERT model) and then save it to Vector DB with metadata (session information, time, etc.). When it is necessary to retrieve the context, the system queries Vector DB to find the vectors closest to the current message, thereby retrieving related old conversations. This mechanism ensures that the chatbot has an effective **long-term memory** without having to increase the prompt size beyond the limits of the language model.

- **AI model and third-party integration:** The system integrates OpenAI's **GPT-4 language model** (or GPT-3.5 depending on configuration) via API, which acts as the main content generation **"brain"** for the chatbot. GPT-4 was chosen for its ability to understand context and generate very natural responses, suitable for advanced dialogue requirements. In addition, the open architecture allows for easy integration of other AI models when needed. For example, open source models (such as GPT-J, LLaMA 2, etc.) can be used via the HuggingFace Transformers library to run internally to reduce costs or increase security (for example, running a small local model to analyze emotions instead of calling an external API). In addition to GPT-4, the system uses **Sentence-BERT** (via the Transformers library) for the task of generating vector embedding of sentences. Some traditional machine learning algorithms are also used: for example, **k-means clustering** to analyze conversation logs (to detect popular topics that users mention on the admin dashboard), or a simple sentiment classification model from scikit-learn to recognize user moods. All these models and libraries are integrated as separate **services** or **modules** in the backend, communicating with the main service via internal APIs.
- **Deployment infrastructure:** The system's microservices architecture is **packaged using Docker** - each backend service runs in an independent container, including components such as Node.js service, Python service (if any for ML models), database, vector DB, etc. Using Docker ensures a consistent execution environment between machines and between the development environment and the production environment. To manage these containers in the production environment, the system uses **Kubernetes (K8s)** . Kubernetes helps automate the deployment, scaling and management of containers: for example,

when the number of users increases, K8s can automatically clone more containers of the chat service to balance the load; if a container fails, K8s will automatically restart (self-healing mechanism) to ensure the service is always available. The database systems (PostgreSQL, MongoDB, Redis, Vector DB) can be deployed through the corresponding cloud-managed services (e.g. AWS RDS for PostgreSQL, MongoDB Atlas for Mongo, AWS ElasticCache for Redis, Pinecone cloud for Vector DB) to ensure **high security and performance** as well as reduce operational effort. This architecture allows flexible deployment on multiple infrastructures (AWS, GCP, Azure or even on-premises) depending on the needs.

- **Communication and security:** The frontend application communicates with the backend entirely via **HTTPS** (HTTP Secure) so that all transmitted data is encrypted. Users need to log in and authenticate (using OAuth2 or JSON Web Token – JWT) to ensure that only valid users can access the API and open chat sessions. For the realtime WebSocket channel, a **wss** (WebSocket Secure) connection is also used, combined with token checking during the handshake session for protection. Regarding sensitive data, the system applies a layer of security and privacy: user profiles and chat log content are encrypted when stored in the database; at the same time, before sending any user content to a third-party API (such as OpenAI), the system anonymizes the information (removing names, emails, or personal identifiers) to comply with data protection regulations (e.g. GDPR). Users are also given control over their data – they can review, download, or request deletion of their chat history at any time. These measures are intended to build **trust** in users that interactions with chatbots are private and secure.

3.3 Database

The system database design follows a **multi-tier model** to accommodate the different types of data used. Specifically:

- **Relational Database (PostgreSQL):** Stores core information with a clear structure and requires high integrity. Includes tables such as **User** (user information: account, hashed password, email, registration date, etc.), **PersonaProfile** (personalization settings such as virtual character name, gender, assumed age of the chatbot if any, preferred tone, etc.), **Settings** (application configuration for each user), and **Feedback/Rating table** (if the user rates the quality of the chatbot after each session). PostgreSQL ensures constraints and relationships so that data is not inconsistent, and can use **SQL** to query and aggregate reports (e.g. number of active users, average satisfaction score, etc.).
- **Non-relational database (MongoDB):** Stores unstructured or flexible structured data. The main component is **the conversation log collection** : each document in this collection can represent a chat session or an interaction depending on the design, containing the content of the user and chatbot messages, timestamps, as well as contextual attributes (e.g. session ID, analyzed sentiments). In addition, MongoDB can also store **detailed** user persona profiles in JSON format (including lists of personal interests, lists of commonly used words, short

biographies provided by users, etc.). MongoDB is chosen because each user can provide a heterogeneous amount of information; storing as a document allows for easy addition/removal of information fields without affecting others, and is convenient when needing to retrieve a person's entire conversation history.

- **Redis (Cache):** Serves as **a cache** for recent conversation sessions. Redis stores, for example, the last 50 messages of each conversation in the server's memory, making repeated requests or short-term context retrieval very fast (due to accessing RAM instead of disk). Redis can also be used to store session **tokens** if JWT is not desired, or to store **temporary statistics** during processing that do not need to be fed into the main database. Data in Redis often expires after a period of time to avoid long-term memory usage.
- **Vector Database:** This is a special storage mechanism for embedding **vectors** for semantic search. Each user's conversation will be mapped into a 768-dimensional vector (if using SBERT), for example, and stored in Vector DB with identification information (such as ConversationID, MessageID). When needed, the system will perform a **nearest neighbors query** on Vector DB to find vectors similar to the vector of the current message. The returned results are previous conversations with similar content. These segments are then extracted from MongoDB (based on the ID stored with the vector) and fed into the prompt to remind the context for GPT-4. Vector DB can be deployed using specialized services such as *Pinecone* (cloud service for vector search) or *Milvus* , or even using the *FAISS library* combined with a regular database (FAISS + SQLite/Postgres) for persistent storage. Separating Vector DB helps to optimize **semantic search processing, high query speed even when the vector set is large with millions of elements.**

Thanks to the coordination of the above database components, the system ensures that data is stored and retrieved efficiently **and** in **context** : relational data is secure and organized, conversational data is flexible and fast to access, vector data is specialized for the purpose of semantic search, and caches speed up immediate tasks.

3.4 Processing Flow

The process of processing an interaction between a user and a chatbot takes place through several consecutive steps in the system. Below is **an overview of the processing flow** for a user message sent to a chatbot:

1. **User types message & sends:** On the client application (mobile app), the user types his message and presses send. The application will transfer this message (with user identification information, chat session) to the backend server via API or WebSocket.
2. **Chat Service receives messages:** On the backend, the Chat Service receives a new message. The service creates a **“message request” object** that includes the message content, userID, conversationID, etc., and starts processing. First, it quickly saves the message content to **MongoDB** (temporary log) and Redis (short-term memory). Then, the Chat Service sends an internal request to **the Memory service** (memory module) to retrieve the relevant context.

3. **Retrieving context from memory:** The Memory Service will take care of finding relevant past information. It checks **the Redis cache** for recent messages (short-term context). At the same time, it creates a vector embedding for the current message (using SBERT) and then queries **Vector DB** to find the most similar vectors – i.e., find old conversations with relevant content. The relevant results (e.g. top 3 old conversations) will be retrieved from **MongoDB** and returned to the Chat Service. Thus, the Chat Service receives a set of context data including: previous messages in the conversation (if any) and important information from previous sessions (if relevant to the current topic).
4. **Generate prompts for the language model:** The Chat Service passes context data to **the Prompt Builder service** . The Prompt Builder's job is to combine the current user message with the retrieved context and **persona instructions** to form a complete input prompt for the language model. Specifically, the Prompt Builder will build a text string that includes: (a) system instructions that describe the role of the chatbot, such as "You are an AI clone of the user, please respond with the most similar tone and emotion to them"; (b) important context content (e.g. "<Context>: [summary or quote of relevant previous information]"); and (c) the **user's most recent message** in an appropriate format (user message). This final prompt is the input sent to the GPT-4 model.
5. **Call LLM model to generate response:** LLM integrated service receives prompt from Prompt Builder and makes call to OpenAI API (or internal language model) to generate response. GPT-4 model will process the prompt, based on context information and persona instructions to generate response in text form. This raw response is returned to Chat Service. Typically, calling GPT-4 via API takes a few seconds, and can use **streaming mechanism** to gradually send part of the response to frontend (similar to how ChatGPT displays gradually).
6. **Post-processing & CBT advice:** Once the response is received from the language model, the Chat Service passes it to **the Response Processing/CBT service** for review and correction if necessary. This module evaluates the content of the response and the user's **sentiment** in the message. If it detects that the user is expressing negative emotions (based on sentiment analysis or keywords, such as "sad", "depressed", or other cognitive distortions), the CBT module can add an encouraging or light-hearted question to the response. For example, the chatbot can add "I understand how you feel. Do you think there is a more positive way to look at this?" at the end of the response, to help the user reframe their thinking in a positive way. The goal is to ensure that the chatbot maintains the user's tone while also providing psychological guidance when appropriate. After this step, the final response is ready.
7. **Return results to the user:** Chat Service sends the final response to the frontend application via WebSocket. The application receives and displays the chatbot's response to the user. Thanks to the real-time mechanism, users receive responses almost immediately after sending the message (delay of only a few seconds). The chat experience is continuous **and natural** .
8. **Update logs and metrics:** In parallel with sending the results, the system updates **the conversation log** . The chatbot's response content is saved to MongoDB as part of the current session log. At the same time, the **Similarity**

service calculates **the similarity** between this response and the user's writing style (by comparing the SBERT vector of the response with the average vector representing the user's text). The similarity score result can be saved for later evaluation. In addition, other metrics such as response time, session length, etc. can also be recorded.

This process happens automatically every time a user messages, ensuring that each chatbot response incorporates the user's **past and present** (current context and previous session knowledge) and **personality** . All the complex processing steps are completed within seconds, providing a conversational experience that feels as fluid as real time.

3.5 KPI

To evaluate the effectiveness and success of the personalized chatbot system, the project sets out a number of **key Key Performance Indicators (KPIs)** as follows:

- **Similarity :** Measures how well the chatbot mimics the user's writing style. Uses the **cosine similarity measure** on the SBERT embedding vector between the chatbot's response and the user's own sample response (or compared to the user's writing style). The KPI target is an average similarity score of **0.85 (85%)** or higher. This metric confirms that the chatbot effectively mimics the user's writing style.
- **Contextual memory:** Measures through specific tests whether the chatbot remembers information from previous sessions. For example, after providing the chatbot with information in a previous session, the user can ask the chatbot for the same information in a subsequent session; if the chatbot answers in detail without the user having to repeat it, it is considered successful. A KPI could be **the rate of correct information retrieval** từ bộ nhớ dài hạn (mục tiêu ≥ 90% trường hợp thử nghiệm).
- **Response time:** Measured from when a user sends a message to when the chatbot displays a response. The goal is to keep the average response time under **3 seconds** per message (of reasonable length). This includes the time spent calling the GPT-4 model via the API and other processing. Real-time experience is important to avoid user disconnection, so speed KPIs are closely monitored.
- **Relevance:** Measures how well the chatbot's responses answer the question and are relevant to the context. This can be assessed through user surveys or expert scoring. Aim for ≥ 90% of responses to be rated as "relevant" or "very relevant" to the situation.
- **User satisfaction:** Although difficult to measure quantitatively, the project sets an indirect KPI through surveying test users about **the satisfaction** or **usefulness** of the chatbot. A 5-point Likert scale can be used, with the goal of ≥ 4/5 average score on criteria such as "chatbot understands me", "chatbot helps me think more positively", "I want to use this chatbot regularly".
- **Stability and reliability:** Through metrics such as system uptime (target > 99%), no crashes or data loss during testing. Additionally, control the chatbot from

generating inappropriate or harmful content (track the number of times content is blocked by filters, target = 0 serious harmful cases).

These KPIs are tracked and measured during testing and limited testing. They help the development team quantify how well the system is meeting its goals, allowing them to adjust or improve specific components (e.g. tweak the prompt if the similarity is not met, improve memory if the chatbot forgets the context, etc.).

3.6 Implementation Tools

The implementation of the chatbot system uses many modern **technologies and tools** that meet the requirements of cross-platform and AI integration. Below is an overview of the main technologies used in the project:

- **Programming Language & Framework (Frontend):** Cross-platform client built with **Dart using the Flutter** toolkit . Flutter allows for write-once and deploy on both Android and iOS with high performance and a smooth interface. The user interface is designed for a simple, intuitive **conversation experience** , and Flutter supports easy display of messages in chat bubbles, scrolling lists, etc. Flutter was chosen for its performance advantages and strong community support, allowing small teams to deploy quickly across multiple platforms. *(In case of needing to expand to the web, Flutter Web or a JavaScript solution like React can be considered, but the current focus is on mobile.)*
- **Language & Framework (Backend):** The server side is developed using **Node.js** (JavaScript/TypeScript) with the **Express.js framework** . Node.js is famous for its non-blocking I/O model, which is very suitable for building real-time applications such as chat servers. Express helps to quickly create API routes as well as manage middleware flexibly. Using JavaScript/TypeScript on the backend also has the advantage of unifying the language with the frontend (if there is a web app using JS in the future). In addition, the Node.js backend easily integrates the **Socket.IO library** to set up a WebSocket channel, allowing real-time data transmission. Socket.IO is used to implement the streaming response feature: the chatbot can send each part of the response as soon as it is available (instead of waiting for the entire response to be sent), helping the interface display the response gradually, word by word, similar to the ChatGPT experience.
- **AI Platform & Machine Learning Library:** The project leverages **the OpenAI API** to access the GPT-4/GPT-3.5 language model. User conversations will be sent to the OpenAI service and receive responses generated by the model. In addition, some other machine learning functions are implemented in **Python** because the Python ecosystem has many powerful ML libraries. Specifically, the development team created a separate Python service that integrates **HuggingFace Transformers** to use the **Sentence-BERT model** to calculate embedding vectors for sentences. The Transformers library provides a built-in SBERT model (Reimers & Gurevych, 2019) to efficiently convert sentences into semantic vectors. Python is also used in conjunction with **scikit-learn** , **NumPy** , **SciPy** to perform log data analysis: for example, calculating cosine similarity,

running k-means to cluster topics that users frequently talk about, or applying TF-IDF to find prominent keywords in user self-reflection content. These modules run separately but communicate with the core Node.js backend via internal APIs (or queues) to return results (e.g. Python module takes a sentence, returns a vector). The multi-language approach allows to use **the strengths of each platform** : Node.js for high request processing speed, Python for rich ML computing capabilities.

- **Database Management System:** The project uses a combination of **PostgreSQL** for relational data and **MongoDB** for document data, as mentioned in the database design section. Both are popular technologies: PostgreSQL v13+ and MongoDB 4.4+ are installed on the server (or using the corresponding cloud service). In addition, **Redis** (v6+) is deployed for caching and **the vector database Pinecone** (or Milvus) is used via API or self-hosted for vector storage. The choice of these tools is based on stability, performance and ease of Node/Python integration (there are official client libraries for Node.js and Python to connect).
- **Deployment & DevOps:** All backend components are containerized using **Docker** . Each service runs in a separate Docker container (e.g., Node.js Chat Service container, Python ML Service container, Postgres container, Mongo container, etc.). Docker Compose can be used in the development environment to quickly orchestrate the entire stack. In the production environment, these containers are managed by **Kubernetes** . The Kubernetes cluster ensures scalability (automatically scaling up/down the number of containers according to load), reasonable resource allocation, and self-healing when the container fails. In addition, the project uses **Git** (GitHub/GitLab) to manage the source code. A **CI/CD process** is established: every time the code is pushed to the repository, the CI pipeline (e.g. using GitHub Actions or Jenkins) will automatically run the build, execute the test suite (if any), then package the new container and deploy it to the Kubernetes cluster. Thanks to CI/CD, the deployment of updates is fast and human errors are limited. Logging and monitoring are also integrated (using tools such as Elastic Stack, Prometheus + Grafana) to monitor system performance and detect anomalies early.

In short, combining the above tools and technologies helps the project develop smoothly, taking advantage of the powerful existing platforms. Choosing Flutter brings a beautiful and consistent interface application on mobile, Node.js + Python in the backend ensures performance and AI integration, and Docker/Kubernetes/Git CI-CD creates a professional deployment process, ready for future system expansion.

IV. Results and Discussion

After the design and development process, the personalized AI chatbot system was deployed as a prototype and initial testing was conducted. **The results** were very positive, showing the effectiveness of the proposed approach:

- Chatbots can **convincingly mimic the user's writing style** . In test scenarios, users were asked to write a sample paragraph for the system to learn their style,

then converse with the chatbot. Analysis showed that the average similarity between the chatbot's responses and the user's text was around **0.87** (87%), higher than the target of 85%. For example, if the user frequently used funny words, emojis, or humorous expressions, the chatbot would respond in a similar tone, making many testers feel like they were talking to "another version of themselves."

- The system demonstrated **good long-term memory** . In conversations that lasted for several days, the chatbot still remembered important details that the user had shared (thanks to the vector memory mechanism). For example, in a new chat session, the user asked again: *"Last time I talked about my plan to change jobs, do you remember the advice you gave?"* , the chatbot responded accurately to the advice given earlier, while summarizing the relevant context. This shows that the RAG solution (combined with vector database) helped the chatbot maintain **a continuous story line** and understanding of the user over time.
- In terms of **speed** , the chatbot responds almost instantly to short messages (1-2 seconds), and under 4 seconds for messages longer than ~100 words. Real-world experience shows that users hardly have to wait, especially with the response displayed in a streaming style, the conversation flows **smoothly** like texting with a friend. The backend system uses GPT-4, so sometimes the speed can be slower than during peak hours, but on average it is still within the acceptable range.
- The quality of the chatbot's response content was assessed **as appropriate and useful** . Through a quick survey of the test group, the majority of users commented that the chatbot understood what they asked and answered to the point. Notably, when users were in a bad mood and sent negative messages, the chatbot not only responded sympathetically but also subtly incorporated **CBT suggestions** to help them see the problem more positively. A specific example: the user complained *"I'm a failure, I can't do anything properly"* , the chatbot responded with the user's own sympathetic tone *"I understand this feeling of disappointment..."* and then cleverly asked *"... am I being too hard on myself?"* . Many testers said that such questions made them stop and rethink the problem - exactly the self-reflection purpose that the system aims for.
- Cross **-platform** and user interface are highly appreciated: the test application runs stably on both Android and iOS, the chat interface is simple but friendly, there is support for displaying logs of previous sessions and a daily self-reflection reminder. Users can review the entire story with the chatbot as a kind of **personal diary** , which helps them realize the process of changing their thinking over time.

Along with the positive results, the development team also discussed some of **the limitations and challenges** that remain with the system:

- While the chatbot does a good job of mimicking the user's writing style, it can still produce somewhat **formulaic** or unnatural responses in situations outside of its known data. This is largely due to the reliance on the GPT-4 model, which is trained on general text. To overcome this, it may be necessary to provide the

chatbot with more examples of the user's style (e.g., allowing the user to add additional articles or social media connections for the system to learn from, if allowed).

- In terms of **cost and dependencies** , the current solution uses a third-party GPT-4 API, which incurs token usage costs and requires a stable internet connection. This may limit the ability to scale to a large user base without the appropriate budget. In the future, the team may consider implementing an in-house (domain-tuned) language model to reduce costs and dependencies, but this will require significant computational resources.
- Privacy **and ethics** are also important. Chatbots encourage users to express their personal thoughts, even their sadness or mental health issues. Therefore, protecting user data is extremely important – any breaches can have serious consequences. The system currently encrypts and anonymizes data, but in the long term, there should be an independent security review and compliance with standards (such as whether to seek HIPAA certification for mental health data).
- Although the chatbot incorporates some basic CBT techniques, it **is not a psychologist** . In focus group discussions, we found it necessary to constantly remind users (e.g. via terms of use or in-app notifications) that it is only a tool to support self-reflection and not a substitute for a professional. The chatbot is not yet able to handle serious psychological crises. In practice, the chatbot also gave somewhat general advice when faced with a very complex problem. This highlights the need to **limit the scope of the chatbot's advice** to a safe level, and recommend that users seek professional help when necessary. The system has also integrated content filtering to prevent bad situations (e.g. if a user mentions the intention to self-harm, the chatbot will give a message recommending immediate contact with a professional and provide a hotline number).

Overall, the results demonstrate the feasibility of a personalized AI chatbot model that supports self-reflection. Test users appreciated the novelty of having the chatbot "take on" their role – creating a sense of empathy and prompting them to reflect on themselves. Technical (long-term memory, speed) and feature (CBT counseling) challenges were addressed relatively well within the scope of the project. However, to move towards a complete product, further improvements are needed to address data, ethical, and feature-expanding limitations, as discussed in detail below.

V. Conclusion and Future Works

Conclusion: The project has successfully designed and built a personalized AI chatbot system that supports self-reflection - a solution that combines **conversational AI technology** and understanding of **personal psychology** . By simulating the user's own style and emotions in the response, the chatbot creates a very **private and deep interactive experience** , completely different from conventional chatbots. The system has realized many new contributions: integrating persona profiles into the language model, implementing long-term conversational memory using vector databases, and integrating CBT therapy into the conversation flow naturally. The test results show that the chatbot achieves the set goals of stylistic similarity, memorability, and usefulness,

opening up the potential for practical applications in supporting users' self-awareness and improving mental health. The project demonstrates that with the right approach, AI can become a powerful **“companion”** , helping individuals understand themselves better and promoting positive changes.

Development direction: From the built foundation, the project has many directions to expand and improve in the future:

- *Enhanced personalization:* Investigate ways **to fine-tune** a medium-sized language model on individual conversation data (with user permission) so that chatbots can internalize personas more deeply than just prompts. An internal language model customized for each user (or group of similar users) can improve consistency and reduce dependency on third parties.
- *Expanding the range of psychological techniques:* Integrate techniques from other psychological therapies (e.g., **dialectical behavior therapy (DBT)** , **mindfulness meditation**) to support a wider range of needs. Chatbots can learn to guide users through breathing exercises, short meditations, or suggested journaling exercises to enhance stress reduction and self-awareness.
- *Clinical evaluation and expert collaboration:* Collaborate with psychologists **or** mental health organizations to clinically evaluate the effectiveness of the chatbot. Conduct larger user studies, gathering qualitative and quantitative feedback on the long-term impact of chatbot use on users’ moods and thinking habits. This will help improve the algorithm (based on real-world data) while ensuring the product is developed in a safe and ethical manner.
- *Improve interface and experience:* Add features **to customize the** chatbot interface (such as avatar, voice if voice is integrated) to increase personalization. Develop additional **web app** or **desktop application versions** so that users can access conveniently on multiple devices. In addition, chatbots can be integrated into existing messaging platforms (Messenger, Zalo) so that users can chat with their "AI clone" right in the familiar application.
- *Optimize performance and cost:* Implement model compression techniques, optimize prompts to reduce the number of tokens, or use a distilled model in cases that do not require the full power of GPT-4 to reduce operating costs. In addition, continue to optimize the system architecture, use more efficient caching to minimize duplicate API calls (e.g. cache embedding results or responses to duplicate prompts).
- *Enhanced security and privacy:* Implement more advanced security measures such as end-to-end encryption for conversation data, two-factor authentication for user accounts, and regular system audits for vulnerabilities. In particular, build a **transparent privacy policy** that allows users to control in detail what data is collected and used to train personas. This helps strengthen trust and meet legal requirements when deployed in real-world international markets.

This project is just the beginning, proving the feasibility of the idea. In the future, with continuous improvements in AI and contributions from experts in many fields, **personalized chatbots that support self-reflection** can become a popular tool, helping millions of people **discover themselves and improve their mental health**

every day. This is a very humane application of AI , where technology is personalized to better serve people.

VI. References

[1] J. Ha, H. Jeon, D. Han, J. Seo, and C. Oh, “CloChat: Understanding How People Customize, Interact, and Experience Personas in Large Language Models,” *Proc. CHI Conference on Human Factors in Computing Systems (CHI '24)* , Honolulu, USA, 2024.

[2] K. K. Fitzpatrick, A. Darcy, and M. Vierhile, “Delivering cognitive behavior therapy to young adults with symptoms of depression and anxiety using a fully automated conversational agent (Woebot): a randomized controlled trial,” *JMIR Mental Health* , vol. 4, no. 2, 2017.

[3] S. Samtani, “Improving workplace well-being in modern organizations: A review of large language model-based mental health chatbots,” 2025. (In publishing).

[4] Y. Wang *et al* ., “Evaluating an LLM-Powered Chatbot for Cognitive Restructuring: Insights from Mental Health Professionals,” *arXiv preprint arXiv:2501.15599* , 2025.

[5] B. Keum, J. Sun, W. Lee, S. Park, and H. Kim, “Persona-Identified Chatbot through Small-Scale Modeling and Data Transformation,” *Electronics* , vol. 13, no. 8, p. 1409, 2024.

[6] Y. Babiya, “How LLMs Use Vector Databases for Long-Term Memory: A Beginner's Guide,” Medium.com, April 26, 2025. [Online]. Available: https://yashbabiya.medium.com/how-llms-use-vector-databases-for-long-term-memory-a-beginners-guide-e0990e6a0a3f

[7] Nexla, “Prompt Engineering vs. Fine-Tuning — Key Considerations and Best Practices,” *Nexla Blog* , 2023. [Online]. Available: https://nexla.com/ai-infrastructure/prompt-engineering-vs-fine-tuning/

[8] M. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *arXiv:1908.10084* , 2019.